

清华大学电子工程系本科生核心课

数据与算法 - 第14讲

算法设计思想 (一)

冯伟 (fengwei@tsinghua.edu.cn)

2024年12月17日





提纲

- 蛮力法
- 分治法
- 减治法
- 变治法
- 贪心法
- 动态规划

提纲

➤ 蛮力法

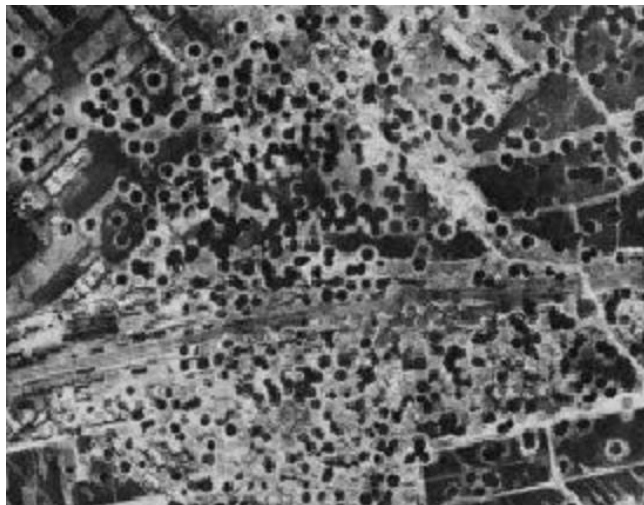
➤ 分治法

➤ 减治法

➤ 变治法

➤ 贪心法

➤ 动态规划



地毯式轰炸



定点击杀本拉登



蛮力法

- 又称为穷举法或枚举法
- 基于问题的描述直接解决问题，无需其他任何技巧
- 思路直接，不考虑算法代价
- 举例：字符串匹配的蛮力法



字符串匹配的蛮力法

穷举匹配：把T的每个位置都作为可能的起始位置，与模式串P逐次进行比较，如相等，则匹配成功，终止；否则就一直比较下去，直到遍历目标串所有位置，匹配失败，过程终止

```
int BruteForceMatch(char *T, char *P){  
    int n = strlen(T);                //求目标串T的长度  
    int m = strlen(P);                //求模式串P的长度  
    int i, j;  
    for(i=0; i <= n-m; i++){          //逐个试探目标串的位置  
        j = 0;  
        while(j < m && T[i+j] == P[j]) j++; //与模式串比较  
        if(j == m) return i;          //在i处匹配成功  
    }  
    return -1;                        //匹配失败  
}
```



字符串匹配的蛮力法

- 假设目标串 T 的长度为 n ，模式串 P 的长度为 m ，蛮力法最坏情况下的时间复杂度为 $O(n \times m)$
 - $T = \text{"aaaa} \cdots \text{ah"}$
 - $P = \text{"aaah"}$
- 优点：直观，实现简单
- 缺点：算法效率不高，只适用于解决小规模的模式匹配问题
- 思考：蛮力法并没有考虑在比较过程中得到的信息



蛮力法

- 蛮力法是一种非常重要的算法设计技术
- 思路很直接，实现简单，也容易被理解（参考教材最大公因子、元素查找、百元买百鸡等其他例子）
- 对规模较小的问题，采用蛮力法是一种很经济的做法
- 一般来说，蛮力法的效率相对较低
- 可作为求解问题的基本方案，用于衡量其它方法的正确性和效率

提纲

➤ 蛮力法

➤ 分治法

➤ 减治法

➤ 变治法

➤ 贪心法

➤ 动态规划

毛主席军事思想：围点打援





分治法 (Divide-and-Conquer)

- Divide: 把问题分解为若干个同属于一个问题的小问题
- Conquer: 重复上面的过程继续分解问题, 如果问题规模足够小, 则直接求解
- Combine: 合并小规模问题的解得到原问题的解
- 举例: 二叉树遍历、快速排序



二叉树的遍历

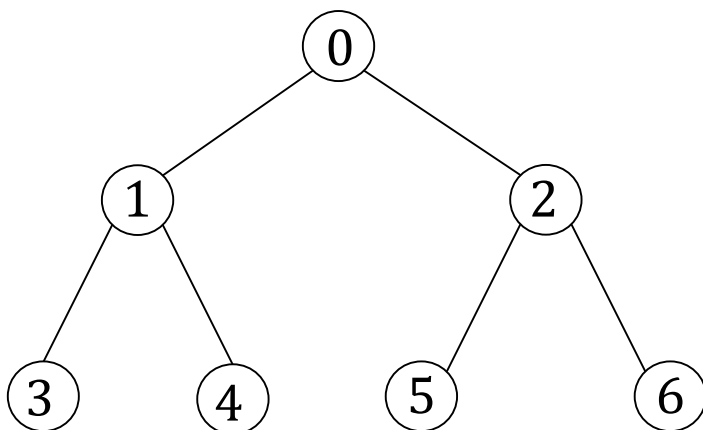
- 二叉树是一种递归结构，可以用递归思路来研究二叉树遍历方法
- 遍历过程对每个结点访问一次且仅访问一次，设访问根结点记作 V ，遍历根的左子树记作 L ，遍历根的右子树记作 R
- 根据访问结点的时机，遍历次序有三种：
 - 前序遍历 $V - L - R$
 - 中序遍历 $L - V - R$
 - 后序遍历 $L - R - V$



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



递归实现

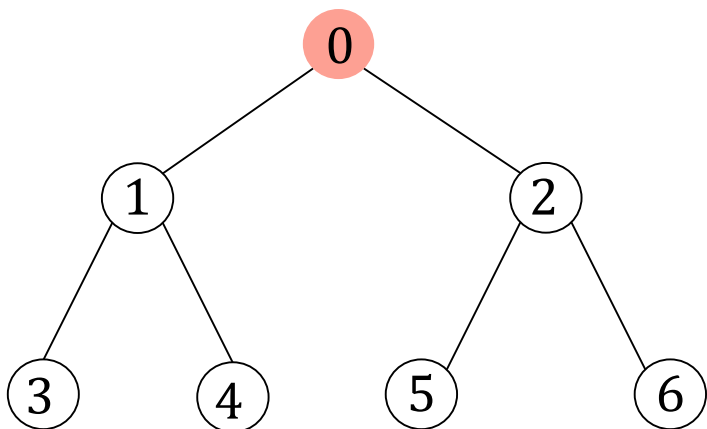
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0

递归实现

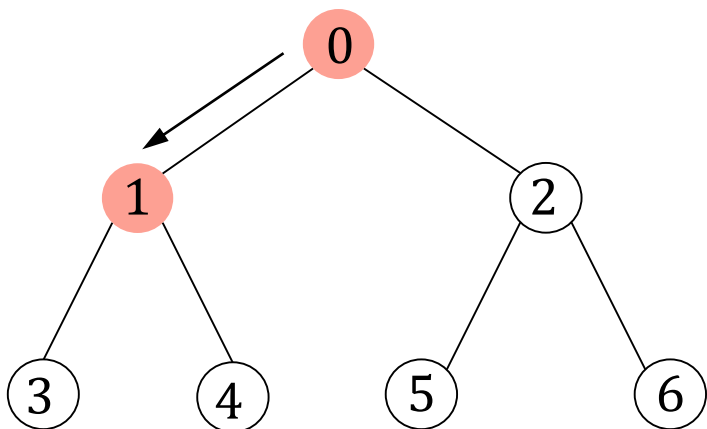
```
void BinaryTree::PreOrder (TNode* t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1

递归实现

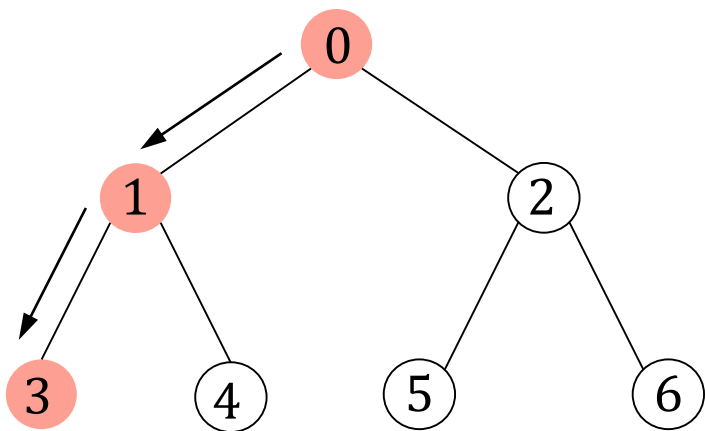
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3

递归实现

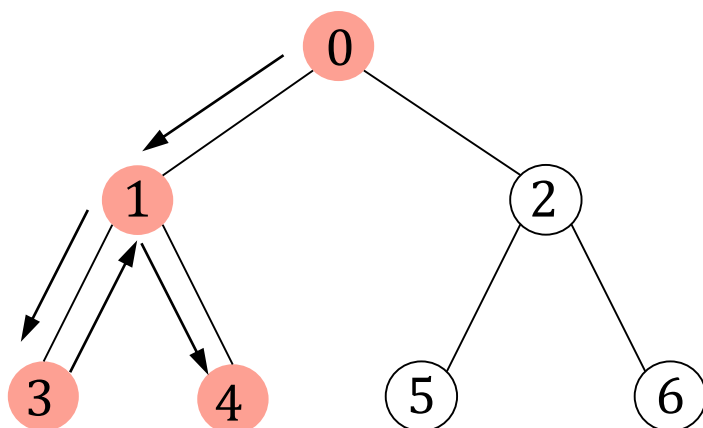
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4

递归实现

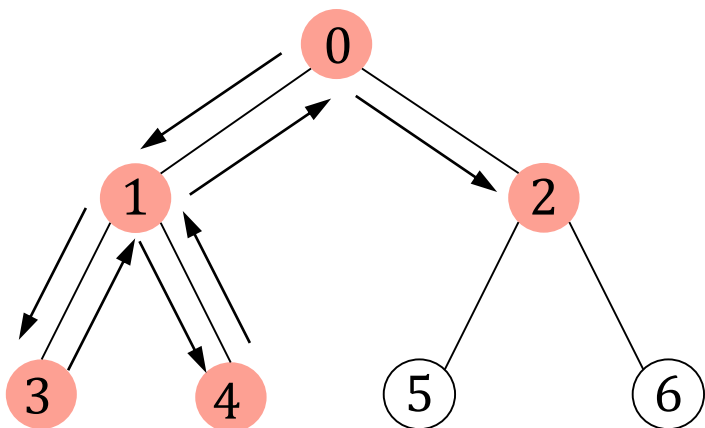
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4-2

递归实现

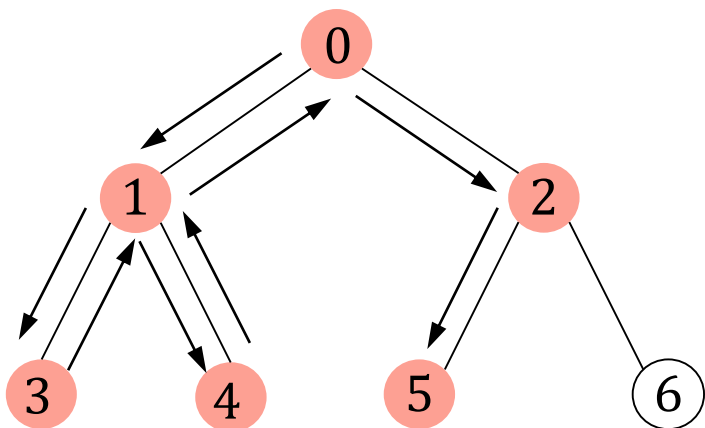
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```




二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4-2-5

递归实现

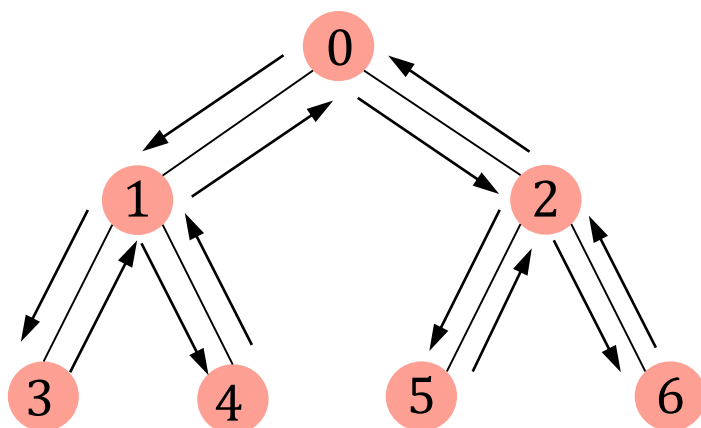
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4-2-5-6

递归实现

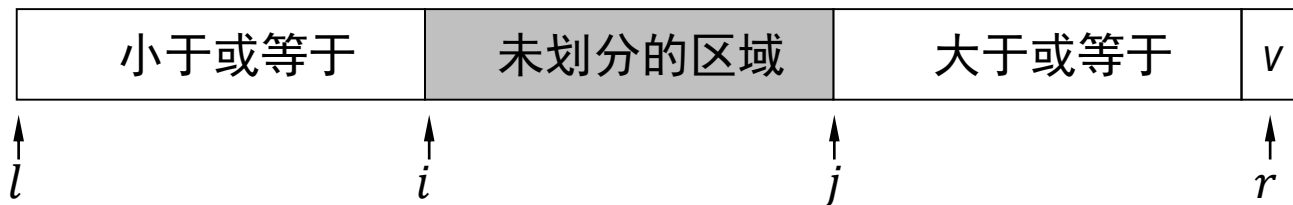
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



快速排序 - 划分

l 和 r 为序列的第一个和最后一个元素， v 表示划分元素的值， i 表示左侧指针， j 表示右侧指针

- 选择 $a[r]$ 作为划分元素-划分后位于正确位置
- 从序列的最左边向中间扫描，找到一个比划分元素大的元素
- 从序列的最右边向中间扫描，找到一个比划分元素小的元素
- 大元素在左，小元素在右，则交换这两个元素
- 当扫描指针 i 和 j 相遇时（可能是划分元素），和划分元素交换，划分完成





快速排序 - 举例

20	26	93	17	77	31	44	55	54
----	----	----	----	----	----	----	----	----

20	26	93	17	77	31	44	55	54
----	----	----	----	----	----	----	----	----

自左向右查找

20	26	93	17	77	31	44	55	54
----	----	----	----	----	----	----	----	----

自右向左查找

20	26	44	17	77	31	93	55	54
----	----	----	----	----	----	----	----	----

交换，自左向右查找

20	26	44	17	77	31	93	55	54
----	----	----	----	----	----	----	----	----

自右向左查找

20	26	44	17	31	77	93	55	54
----	----	----	----	----	----	----	----	----

交换，自左向右查找，相遇

20	26	44	17	31	54	93	55	77
----	----	----	----	----	----	----	----	----

交换



代表划分元素位置



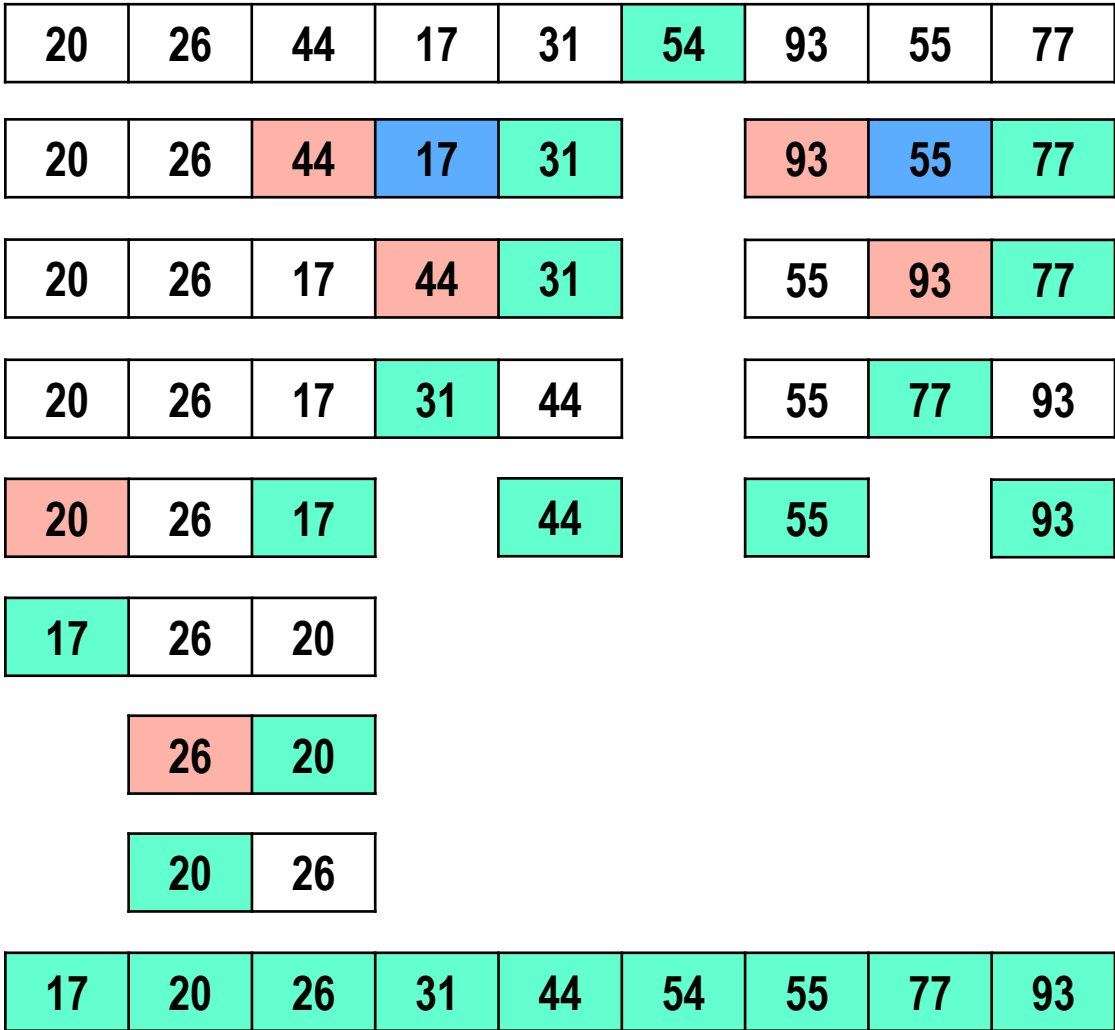
代表左指针位置



代表右指针位置



快速排序 - 举例





提纲

- 蛮力法
- 分治法
- 减治法
- 变治法
- 贪心法
- 动态规划



减治法

- 与分治法思想类似，但略有不同
- 把问题转化为规模较小的子问题，通过求解子问题来得到原问题的解
 - 减治法每次都减小问题的规模
 - 一般只需要处理子问题中的一个
- 举例：选择排序（规模-1）、折半查找（规模缩降）、黄金分割搜索算法



选择排序

逐位确认 → 最小的在第一个、次小、次次小

- 在 $a[i] \sim a[n - 1]$ 中选择最小的元素
- 若不是其中的第一个元素，则与第一个元素交换
- 在 $a[i + 1] \sim a[n - 1]$ 中重复上述步骤，直到仅余一个元素

5	3	8	4	6
5	3	8	4	6
3	5	8	4	6



选择排序 - 评价

- 比较次数与序列初始排列无关，需要 $n(n - 1)/2$ 次比较操作
- 元素移动次数与待排序列初始排列有关：0， $n - 1$
- 优势：实现简单，执行时间比较固定
- 适合元素规模大，用于比较的关键字规模小的序列，比较代价小（仅关键字），移动代价大（整个元素）
 - 例如学生信息（姓名、学号、班级、成绩等）
- 不稳定
 - 非相邻元素交换，关键字相同元素的相对位置在排序中不能保证



选择排序 - 评价

5 (A)	6	5 (B)	2	3
2	6	5 (B)	5 (A)	3
2	3	5 (B)	5 (A)	6
2	3	5 (B)	5 (A)	6
2	3	5 (B)	5 (A)	6
2	3	5 (B)	5 (A)	6

➤ 不稳定

- 非相邻元素交换，关键字相同元素的相对位置在排序中不能保证



线性查找表 - 折半查找

- 如果顺序表对于关键字是有序的，可采用折半查找（对分查找）
- 可用二叉树描述折半查找过程：折半查找树

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

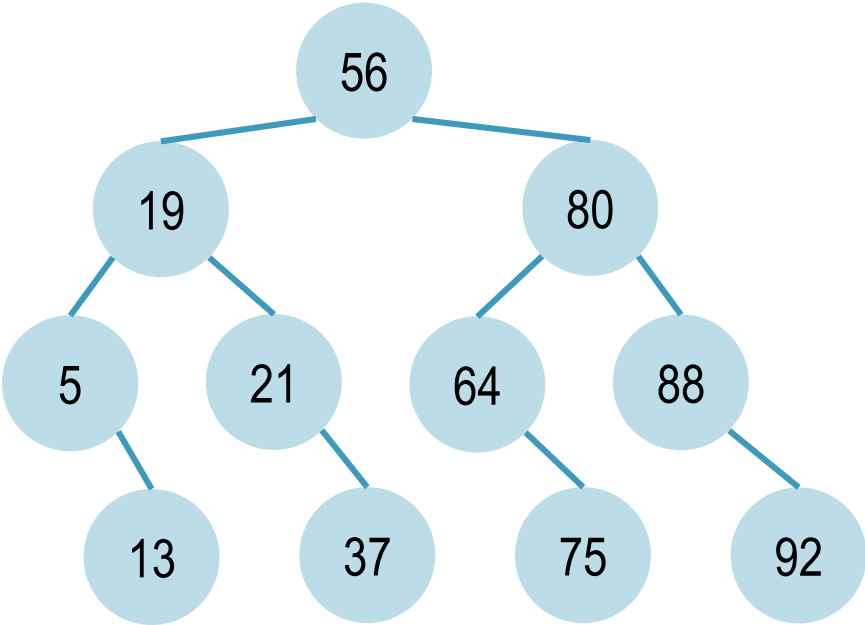
- 起始位置: $left$
- 终止位置: $right$
- 中间位置: mid
- $l = 0, r = 10, m = 5, \dots$



线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

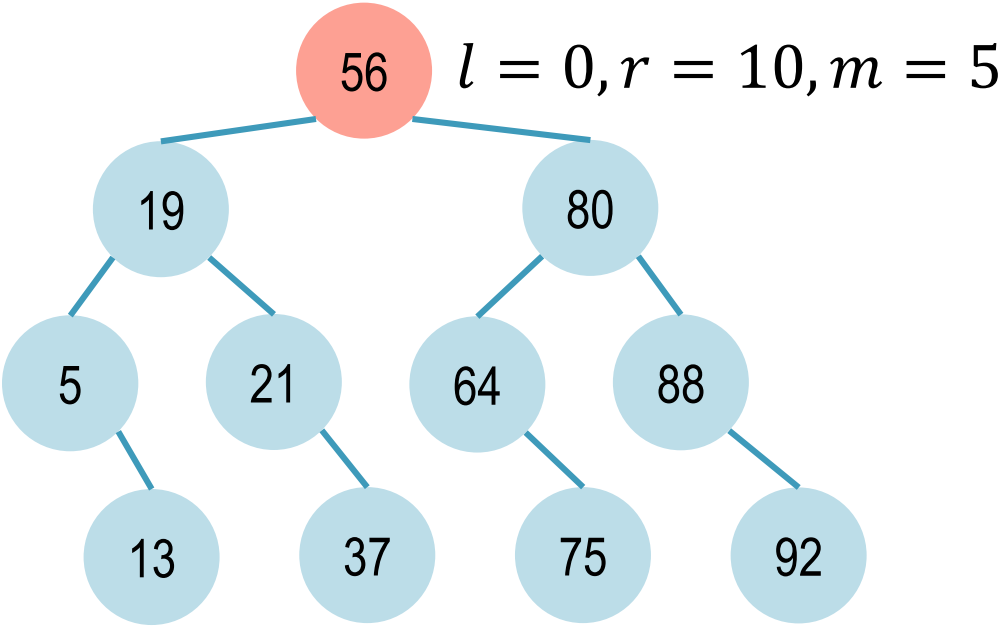




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

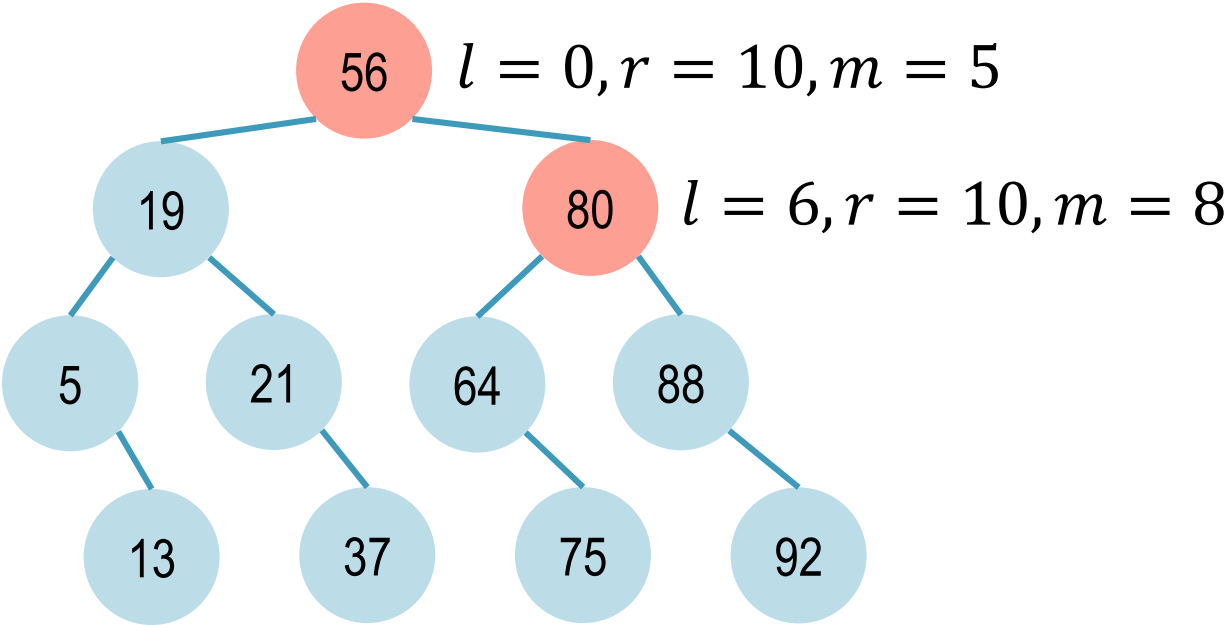




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

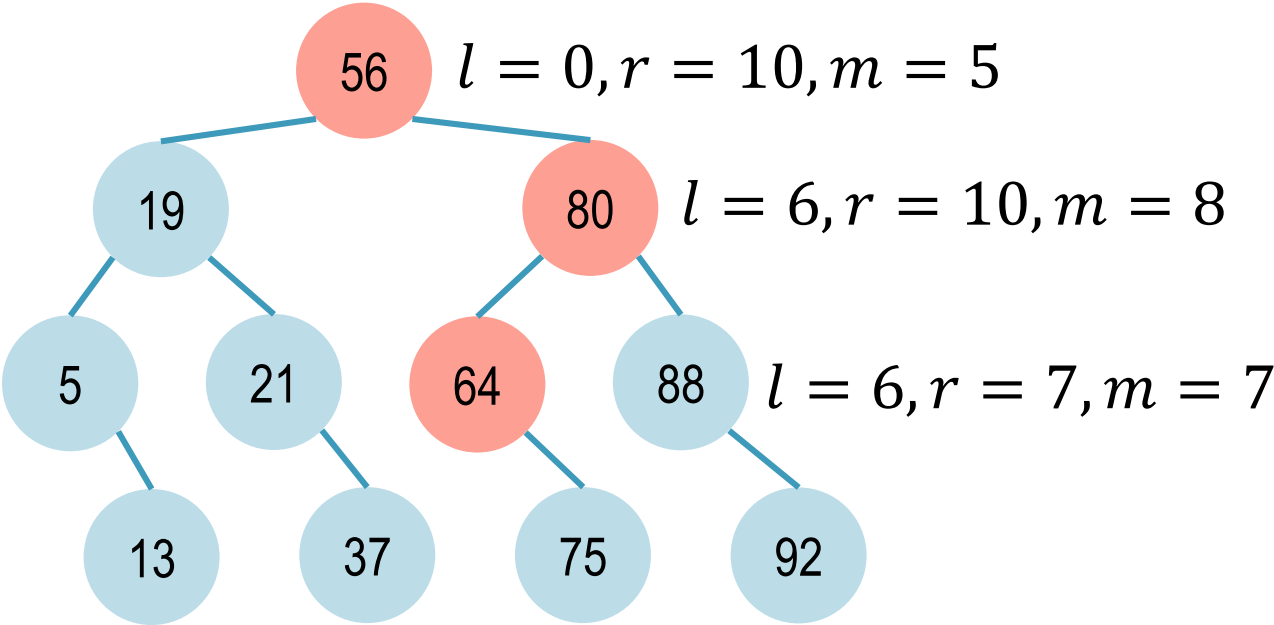




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92

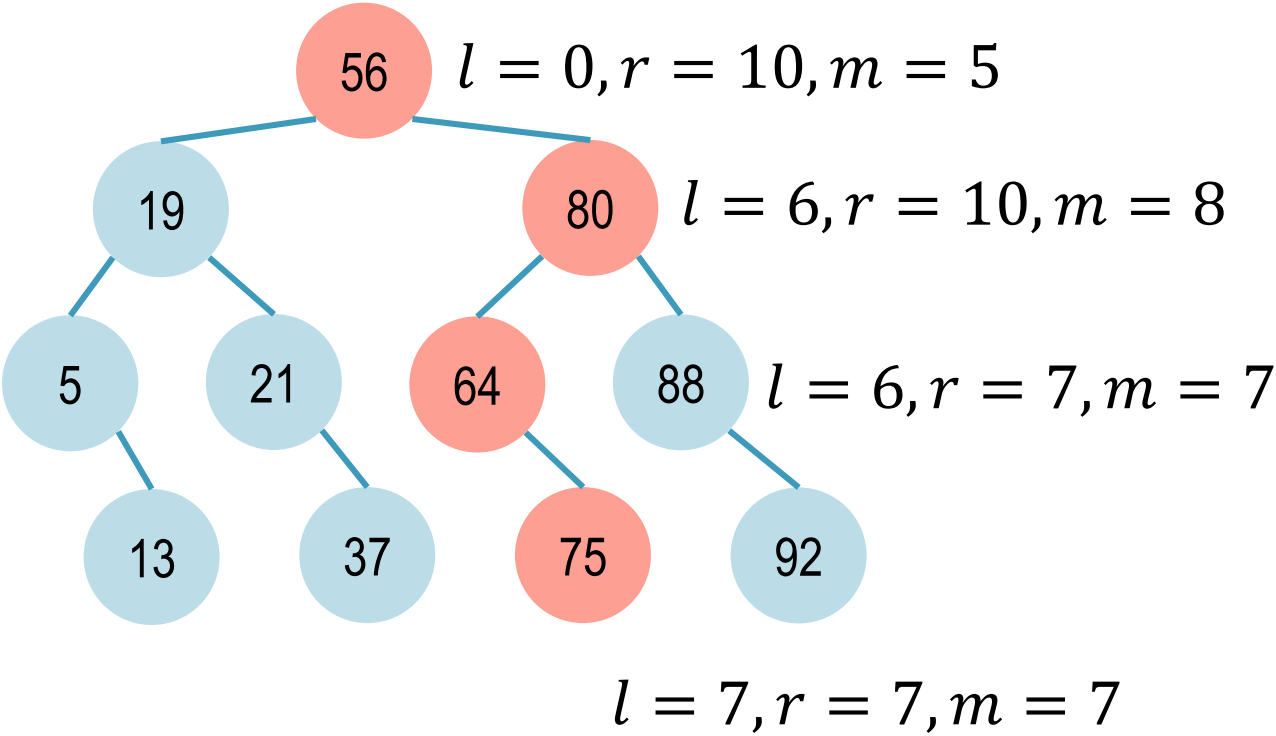




线性查找表 - 折半查找

查找75

0	1	2	3	4	5	6	7	8	9	10
5	13	19	21	37	56	64	75	80	88	92





线性查找表 - 折半查找

- 折半查找可以用一个二叉树结构来描述
- 二叉树的深度是折半查找需要的比较次数
- 对于一次查找，无论成功还是失败，折半查找需要的比较次数不会超过 $\lfloor \log_2 n \rfloor + 1$
- 折半查找的平均查找长度 $O(\log_2 n)$
- 折半查找属于减治算法



提纲

- 蛮力法
- 分治法
- 减治法
- 变治法
- 贪心法
- 动态规划



变治法

- 把求解困难的问题转换成有已知解法的问题，并且转换的复杂度
不高于求解目标问题的复杂度
- 例如：线性表可以排序后再折半查找
- 例如：线性方程组求解
 - 高斯消去法：方阵 $\rightarrow$ 上三角阵
 - LU分解：方阵 $\rightarrow$ 上三角和下三角矩阵



分治法、减治法、变治法

- 分治法通过不断减少问题规模，分而治之，提高效率
- 分治法递归式： $T(N) = a \times T(N/b) + f(N)$ （教材314页）

将规模为 N 的问题分解为 a 个规模为 N/b 的子问题， $f(N)$ 是合并的算法复杂度

- 减治法不断减小问题规模
- 变治法转换问题



提纲

- 蛮力法
- 分治法
- 减治法
- 变治法
- 贪心法
- 动态规划



贪心算法 (Greedy Algorithm)

- 通过一系列步骤来构造问题的解，每一步都是对当前已经存在的部分解的一个扩展，直至获得问题的完整解
- 局部最优：是当前情况下的最优解
- 不可取消：后面的过程不能影响前面的结果
- 贪心算法不能保证对于每个问题都得到最优解
- 一个问题含有最优子结构则该问题最优解中包含子问题的最优解
 - 举例：假如北京 - 哈尔滨必须经过沈阳，可以把最短路径优化问题变成两个子问题：北京 - 沈阳 + 沈阳 - 哈尔滨



霍夫曼树的构造

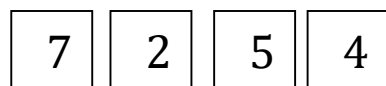
1. 根据给定的 n 个权值 $\{w_1, w_2 \cdots w_n\}$ ，构造 n 棵二叉树的集合 $F = \{T_1, T_2 \cdots T_n\}$ ，其中每棵二叉树中均只含一个带权值为 w_i 的根结点，其左、右子树为空树
2. 在 F 中选取其根结点的权值为最小的两棵二叉树，分别作为左、右子树构造一棵新的二叉树，并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和
3. 从 F 中删去这两棵树，同时加入刚生成的新树
4. 重复2和3两步，直至 F 中只含一棵树为止



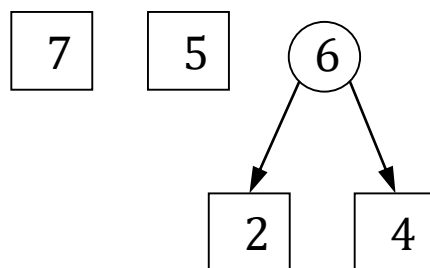
霍夫曼树的构造

一个例子：（具体代码实现参考教材90-91页）

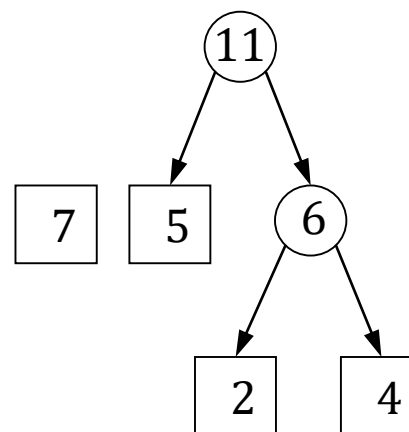
初始
 $F = \{7, 2, 5, 4\}$



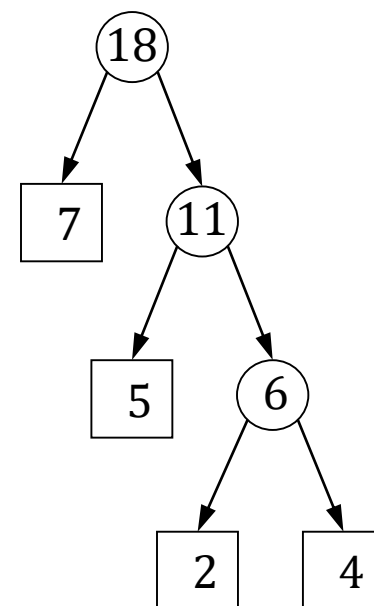
合并2, 4
 $F = \{7, 5, 6\}$



合并5, 6
 $F = \{7, 11\}$



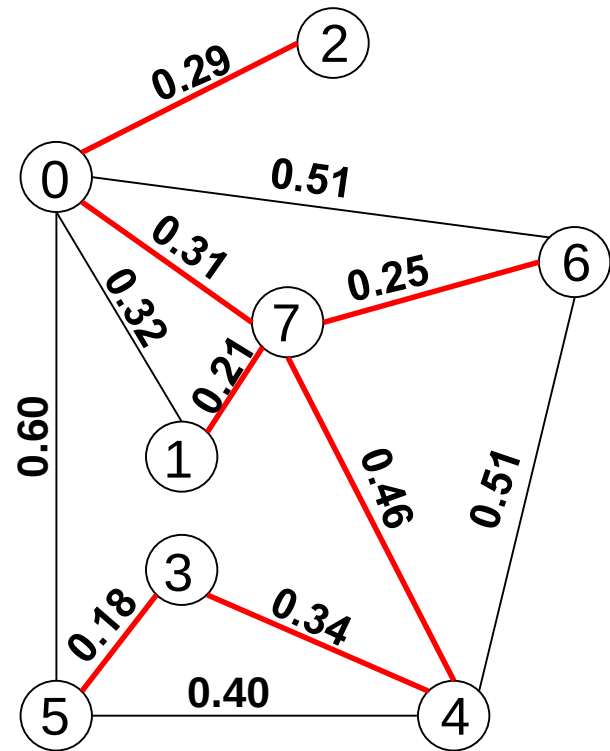
合并7, 11
 $F = \{18\}$





图的最小生成树

- 使用该网络中的边来构造生成树
- 用 $n-1$ 条边来联结网络中的 n 个顶点
- 不能使用产生回路的边
- 权值最小
- 搜索所有可能的 $n-1$ 条边的组合，找权值最小的生成树（蛮力法）
- 在图的顶点、边集合中，从无到有，长出一颗最小生成树
 - 逐顶点
 - 逐边

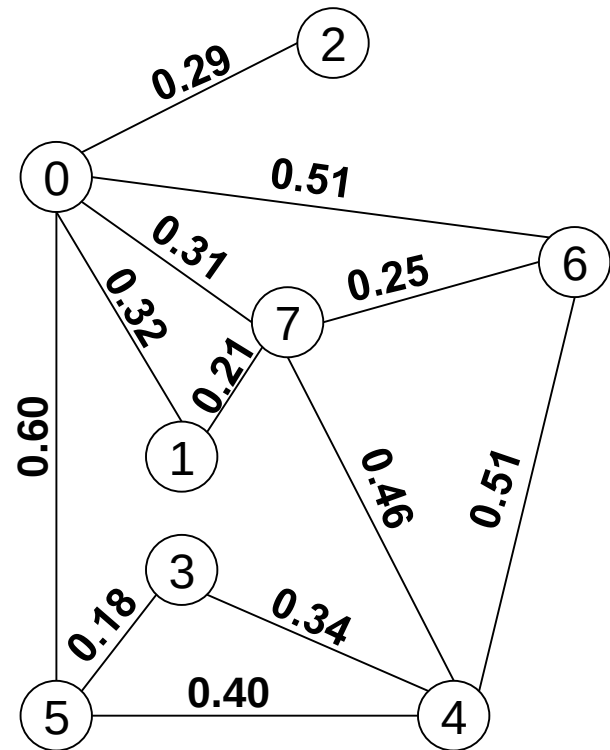


Prim算法、Kruskal算法



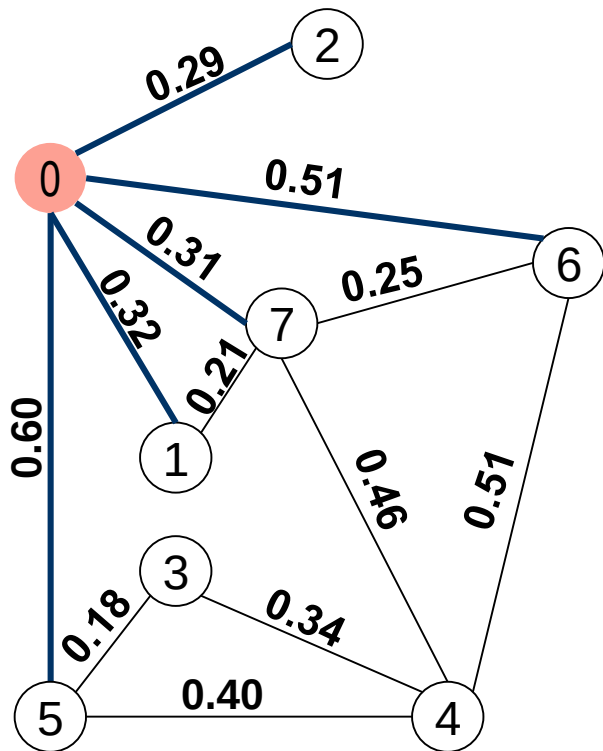
最小生成树的构造算法 - Prim算法

- 初始 MST 是一个空集
- 首先从图 G 中任取一个顶点 a 加入 MST ,
 $MST = \{a\}$, 找出当前非 MST 的顶点
与当前 MST 直接相连的所有边
- 取出最短边, 把它和非 MST 顶点 b 加入
 MST , $MST = \{a, b\}$, 更新边序列
- 重复上述过程, 直到 MST 包括 G 中所有顶点

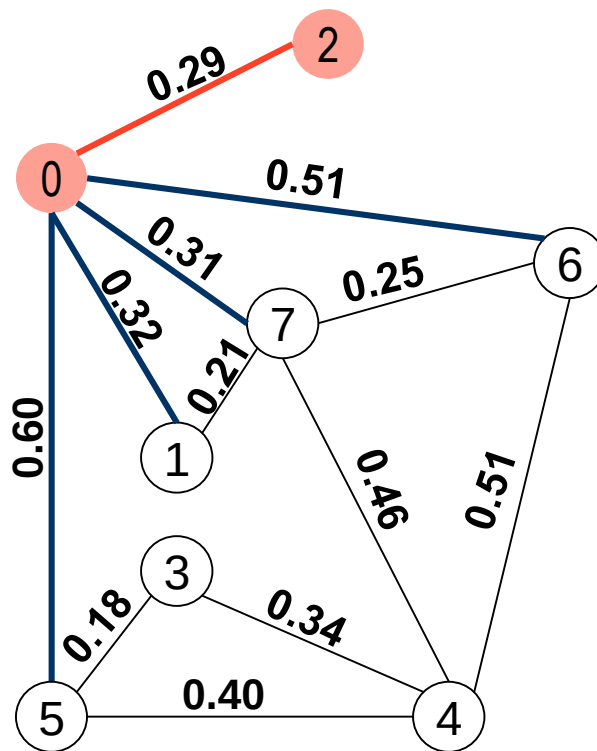




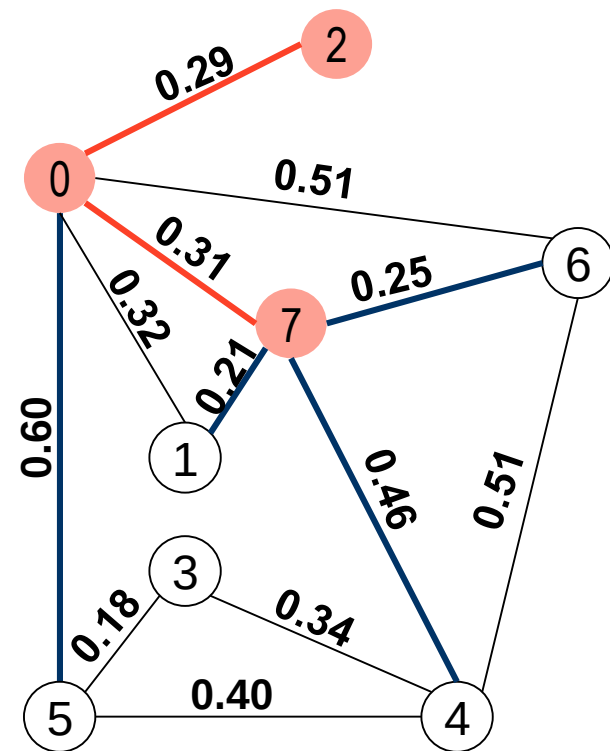
最小生成树的构造算法 - Prim算法



MST = {0}



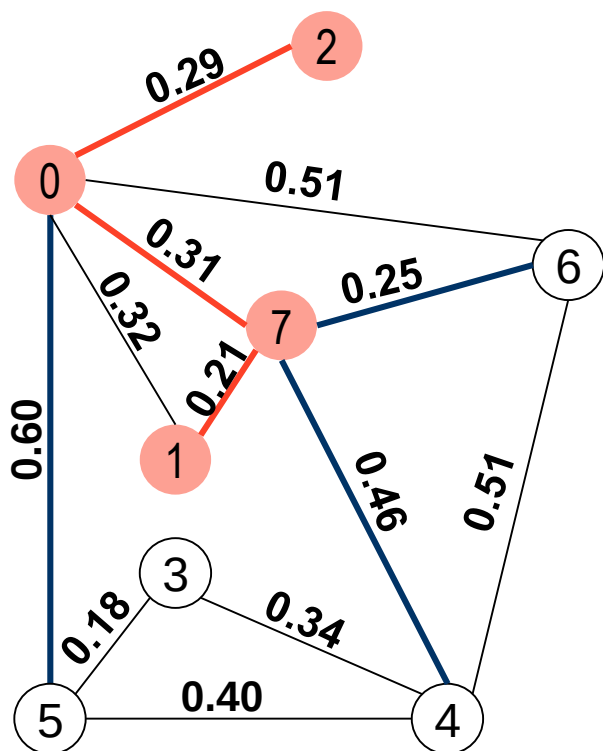
MST = {0, 2}



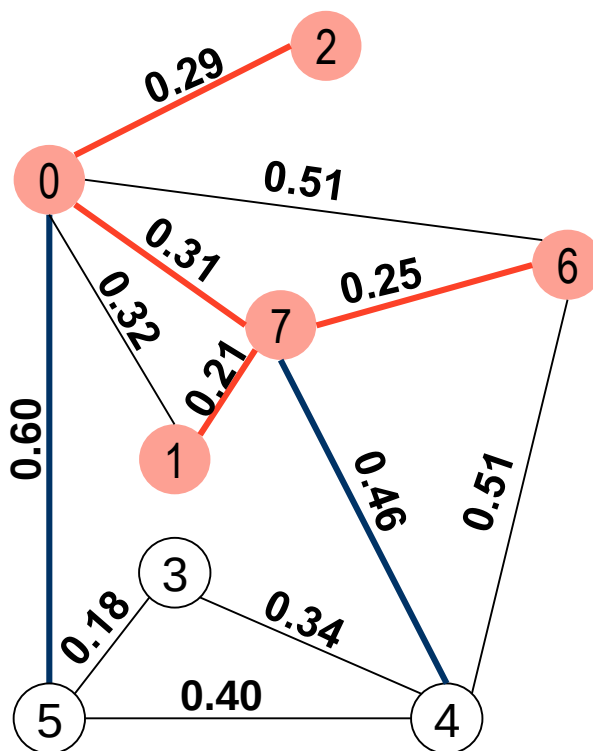
MST = {0, 2, 7}



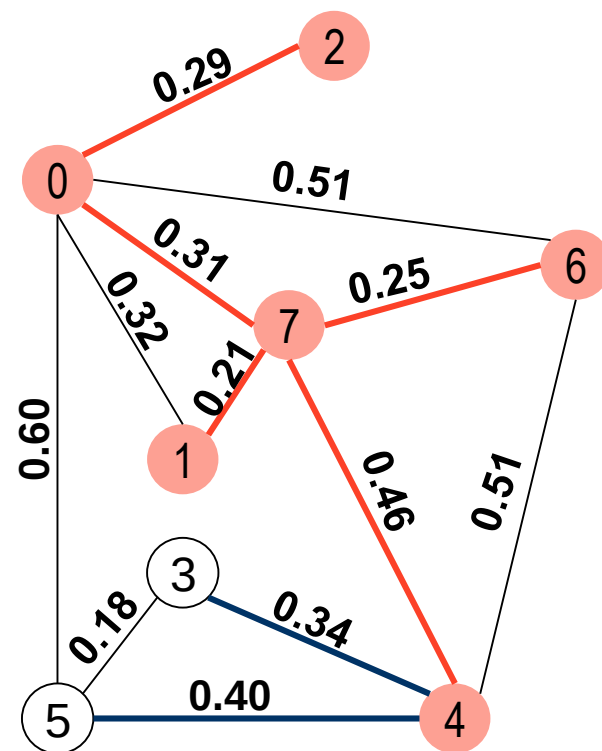
最小生成树的构造算法 - Prim算法



MST = {0, 2, 7, 1}



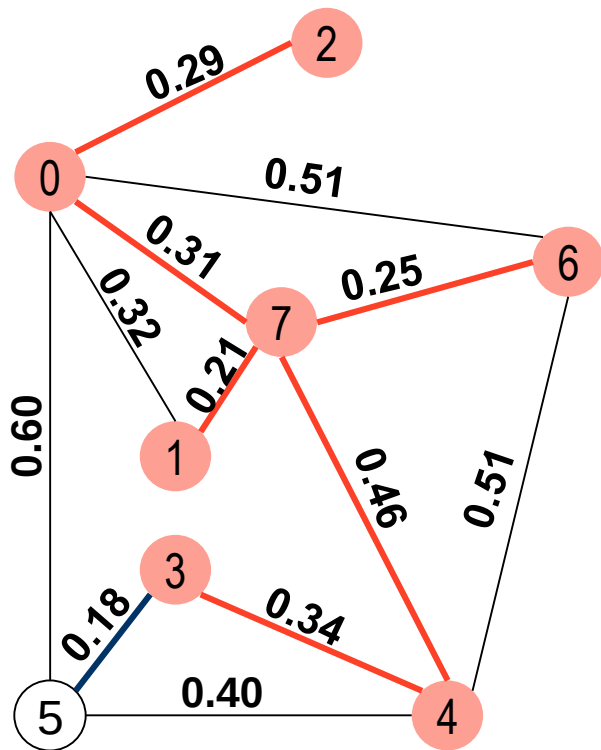
MST = {0, 2, 7, 1, 6}



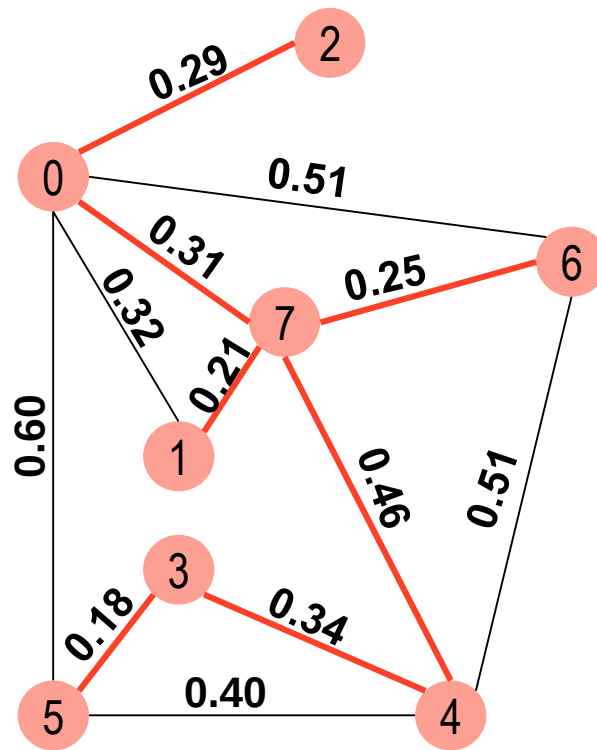
MST = {0, 2, 7, 1, 6, 4}



最小生成树的构造算法 - Prim算法



$MST = \{0, 2, 7, 1, 6, 4, 3\}$



$MST = \{0, 2, 7, 1, 6, 4, 3, 5\}$



最小生成树的构造算法 - Prim算法

- Prim算法将图分成两部分， MST 和非 MST ，每次通过寻找连接两部分的最短边得到距离 MST 最近的顶点，并把最短边和这个顶点加入 MST ，直到 MST 包含了所有顶点
- 程序实现思路：
 1. 建立辅助数组，记录各个非 MST 顶点到 MST 的最短距离
 2. 向 MST 加入距离最小的顶点和对应边（参考教材p141的minEdge函数）
 3. 更新辅助数组
 4. 重复步骤2，3，直至非 MST 中无顶点

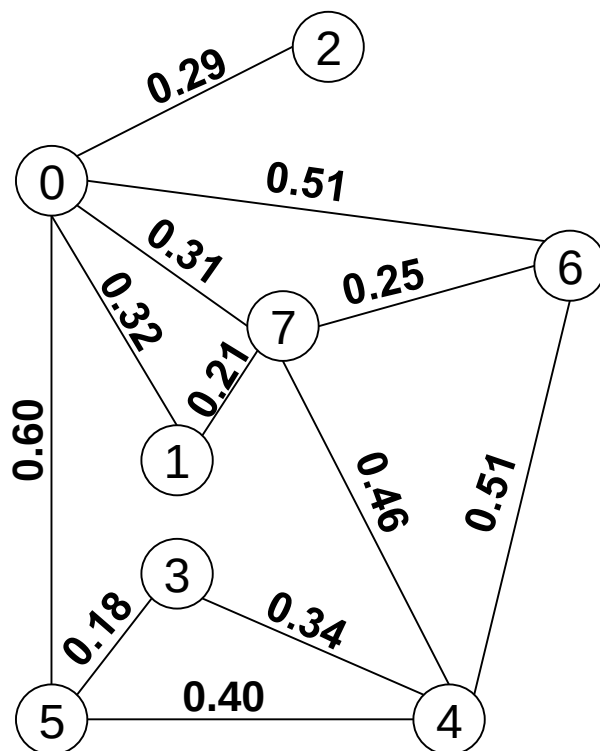
最小生成树的构造算法 - Kruskal算法



- 对图中所有边按权值进行排序
- 令 MST 的边集为空
- 从图中取出权值最小的一条边，如果这条边与 MST 中的边能构成环，则舍弃；否则，将这条边加入 MST
- 重复上述过程，直至将 $n - 1$ 条边加入 MST



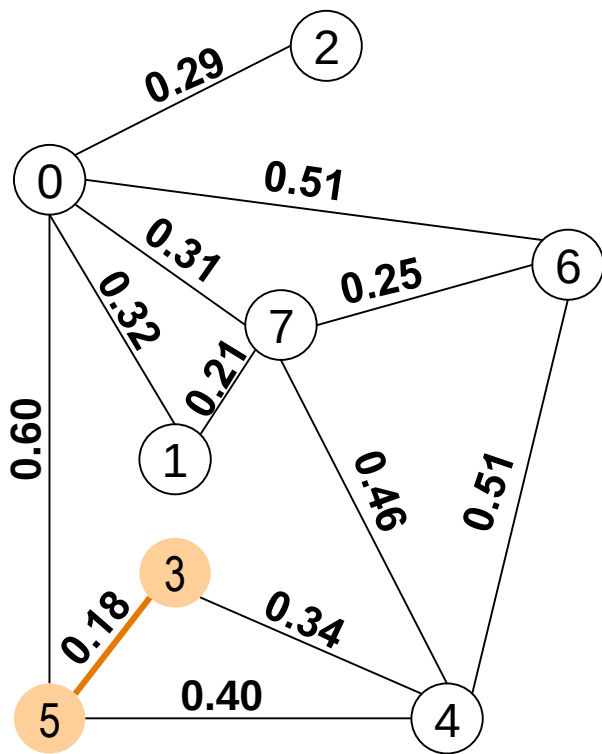
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



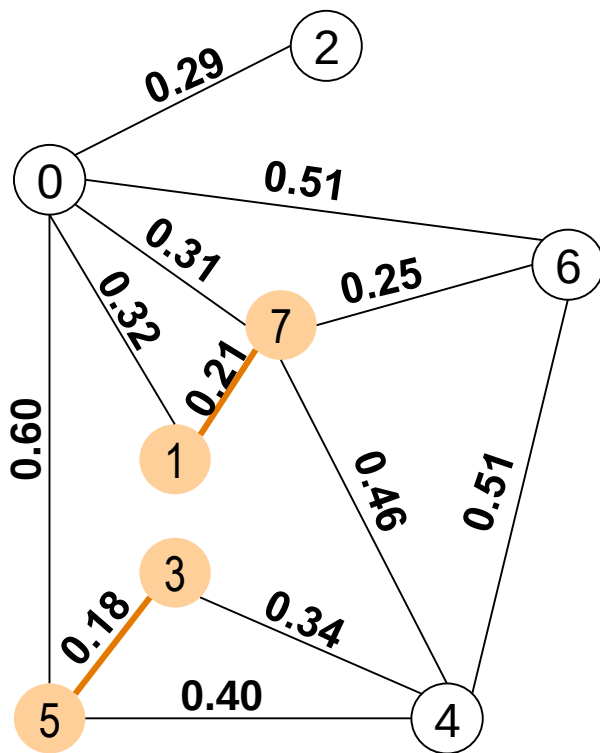
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



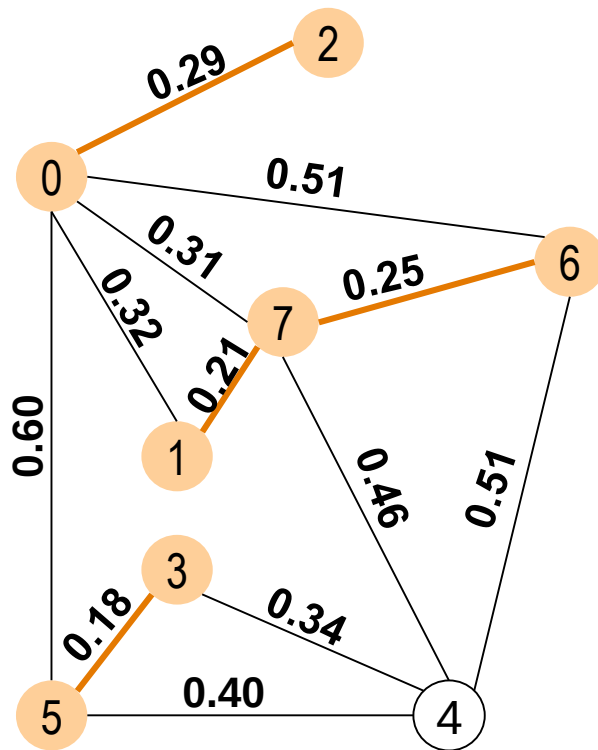
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



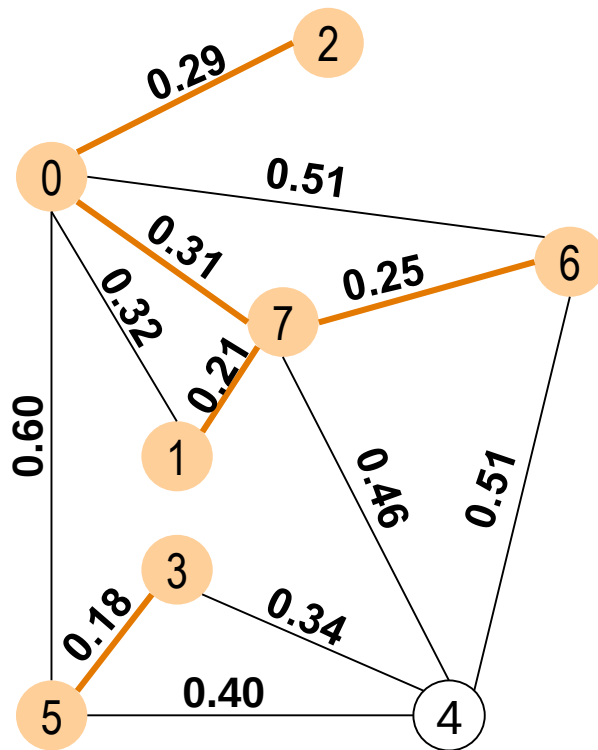
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



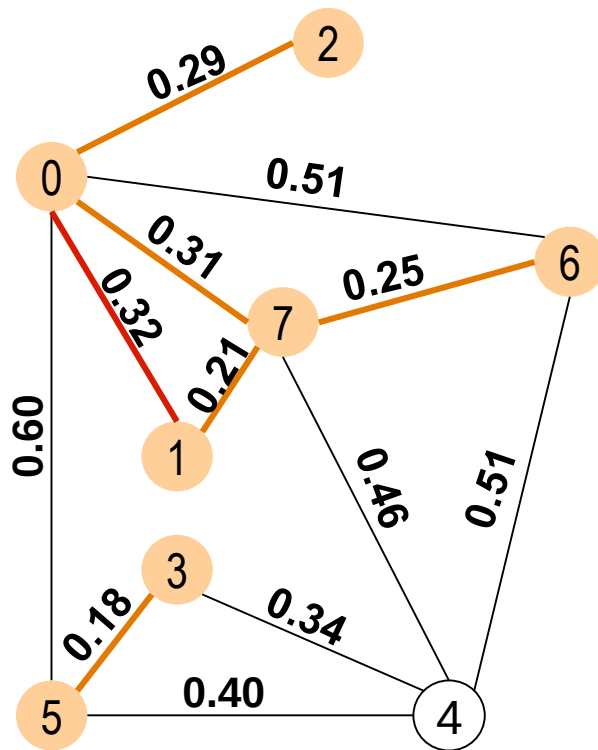
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



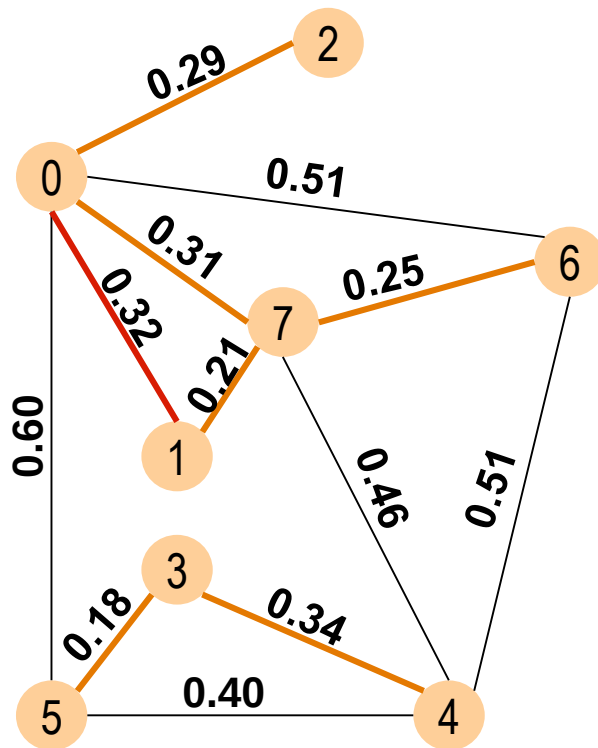
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



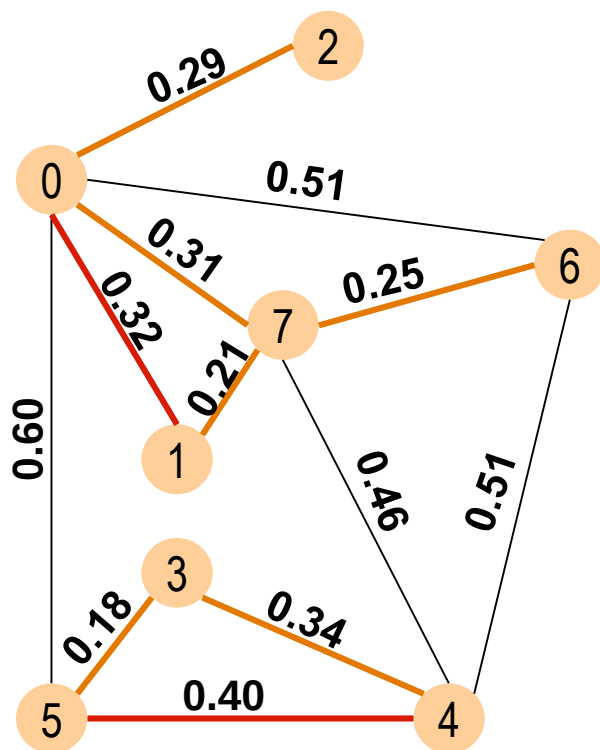
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



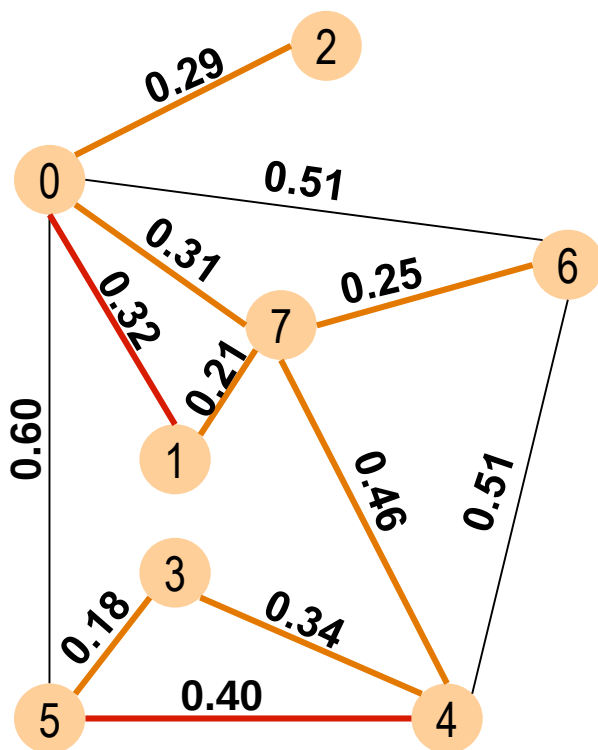
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60

共7条边



最小生成树的构造算法 - Kruskal算法

- Kruskal算法本质上也是贪心算法：每次向 MST 中加入权值最小的边，并保证 MST 中不构成回路
- 在Kruskal算法中，如果找到了 $n - 1$ 条边，则得到最小生成树，如果检查了所有边，而未找到 $n - 1$ 条边，则此图是非连通的
- 算法复杂度
 - 对边的排序，采用高级排序方法，时间复杂度可以达到 $O(e \log e)$
 - 环的判断：维护一个连通关系的等价类集合(教材p143程序示例中的顶点连通子集)，新增边的两个顶点如果在集合里，则加入后会构成环，否则合并入集合，其时间复杂度近乎线性
 - Kruskal算法的时间复杂度为 $O(e \log e)$ ，适用于稀疏图



贪心算法

- 每步做局部优化，有些情况可以得到全局最优解
- 容易设计与实现，计算效率高
- 应用广泛



提纲

- 蛮力法
- 分治法
- 减治法
- 变治法
- 贪心法
- 动态规划



动态规划

- 动态规划将原始问题分解为若干子问题求解，并基于子问题的解构建原始问题的解
- 分治法中各个子问题互不相关，动态规划中各个子问题是相互有关系的
- 各个子问题可以视为求解原始问题的不同阶段，但与贪心法不同，动态规划不仅考虑当前状态，还要考虑其对于后续选择的影响，因此可以获得全局最优解



动态规划

- ① 划分阶段：将原问题划分为相互联系的有顺序的几个环节（在有些问题中，阶段的划分是自然的；但在有些问题中，阶段的划分来自于合理的设计）
- ② 状态：描述当前阶段的进展情况
- ③ 决策：从某阶段的一个状态出发演变到下一个阶段某状态的选择

采用动态规划方法，该问题的最优解应能分解出最优子结构，并满足无后效性条件：未来状态只取决于当前状态和决策，与过去的状态和决策无关



例子：背包问题

序号	1	2	3	4	5	
重量	2	1	3	2	1	w_i
价值	12	10	20	15	8	v_i

- 背包中物品总重量 ≤ 5
 - 寻求价值最大
 - 贪心法：(3, 20) + (2, 15)
 - 蛮力法
- $$\max \sum_{i=1}^n v_i x_i$$
- $$s. t. \sum_{i=1}^n w_i x_i \leq W$$
- $$x_i \in \{0,1\}, 1 \leq i \leq n$$
- n 件物品， W 为承重约束， x_i 为决策变量



背包问题

- 对每个 $i = 0, 1, \dots, n$, 每个整数 $0 \leq w \leq W$ 都存在原问题的子问题
- 用物品 $\{1, \dots, i\}$ 的子集且所允许的最大重量为 w :

$$OPT(i, w) = \max_S \sum_{j \in S} v_j x_j \quad \begin{cases} S \subseteq \{1, \dots, i\} \\ \sum_{j \in S} w_j \leq w \end{cases}$$

- $w < w_{i+1} \rightarrow$

$$OPT(i + 1, w) = OPT(i, w)$$

- $w \geq w_{i+1} \rightarrow$

$$OPT(i + 1, w) = \max(OPT(i, w), v_{i+1} + OPT(i, w - w_{i+1}))$$



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5					
4					
3					
2					
1					
0					
	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)					
4 (2,15)					
3 (3,20)					
2 (1,10)					
1 (2,12)					
0	0	0	0	0	0
	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)						
4 (2,15)						
3 (3,20)						
2 (1,10)						
1 (2,12)	0	0	12	12	12	12
0	0	0	0	0	0	0
	0	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)						
4 (2,15)						
3 (3,20)						
2 (1,10)	0	10	12	22	22	22
1 (2,12)	0	0	12	12	12	12
0	0	0	0	0	0	0
	0	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)						
4 (2,15)						
3 (3,20)	0	10	12	22	30	32
2 (1,10)	0	10	12	22	22	22
1 (2,12)	0	0	12	12	12	12
0	0	0	0	0	0	0
	0	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)						
4 (2,15)	0	10	15	25	30	37
3 (3,20)	0	10	12	22	30	32
2 (1,10)	0	10	12	22	22	22
1 (2,12)	0	0	12	12	12	12
0	0	0	0	0	0	0
	0	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)	0	10	18	25	33	38
4 (2,15)	0	10	15	25	30	37
3 (3,20)	0	10	12	22	30	32
2 (1,10)	0	10	12	22	22	22
1 (2,12)	0	0	12	12	12	12
0	0	0	0	0	0	0
	0	1	2	3	4	5

承重约束



背包问题

序号	1	2	3	4	5
重量	2	1	3	2	1
价值	12	10	20	15	8

物品

5 (1,8)	0	10	18	25	33	38
4 (2,15)	0	10	15	25	30	37
3 (3,20)	0	10	12	22	30	32
2 (1,10)	0	10	12	22	22	22
1 (2,12)	0	0	12	12	12	12
0	0	0	0	0	0	0
	0	1	2	3	4	5

承重约束



全源最短路径问题

- 求出图中任意两个顶点间的最短路径
- 可以对每个顶点应用Dijkstra算法，时间复杂度为 $O(n^3)$
- 另一种算法：弗洛伊德(Floyd)算法，体现动态规划的思想，时间复杂度也为 $O(n^3)$



全源最短路径 - Floyd算法

- 初始化 $n \times n$ 的矩阵 D_0 （即邻接矩阵），为图的边值矩阵
- 按照下面公式进行迭代

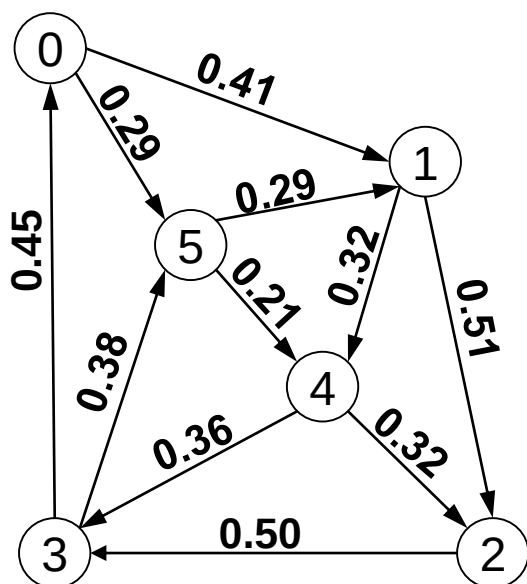
$$D_k[i][j] = \min\{D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j]\}$$

$$k = 0, 1, \dots, n - 1$$

- $D_k[i][j]$ 即为从顶点 i 到 j 的最短路径长度，这条路径不经过编号大于 k 的顶点
- 经过 n 次迭代，最后得到全源最短路径矩阵



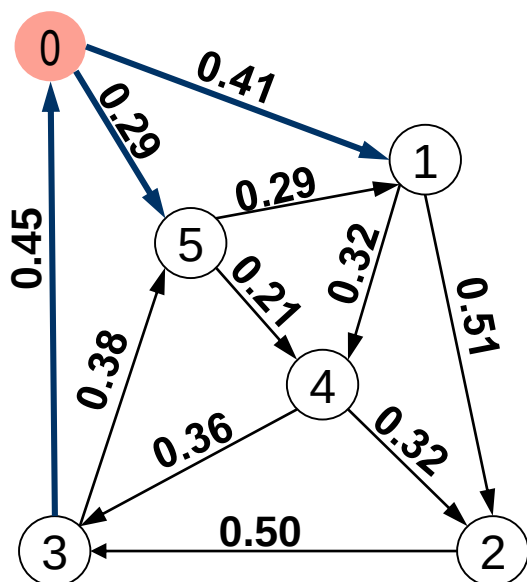
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	∞	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



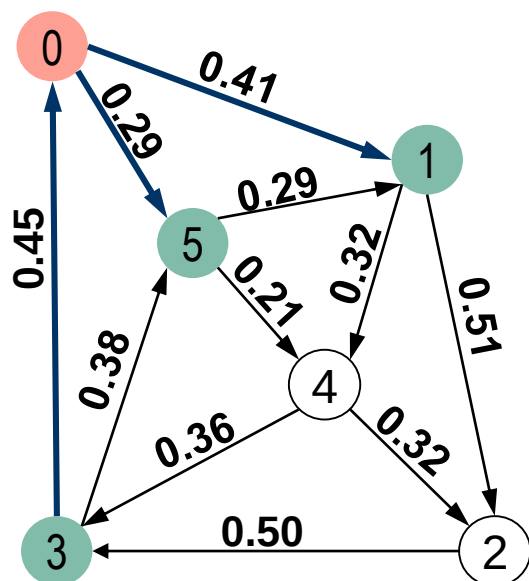
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	∞	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



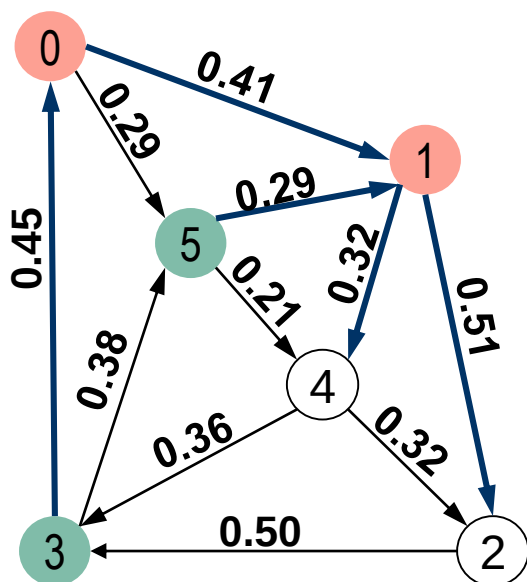
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



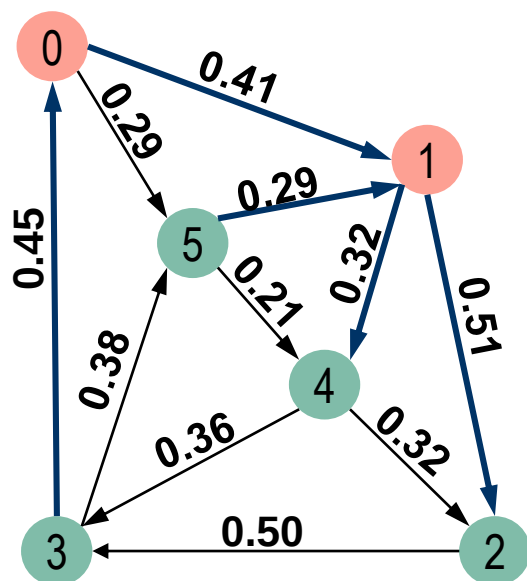
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	∞	∞	∞	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	∞	0	∞	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	∞	∞	0.21	0



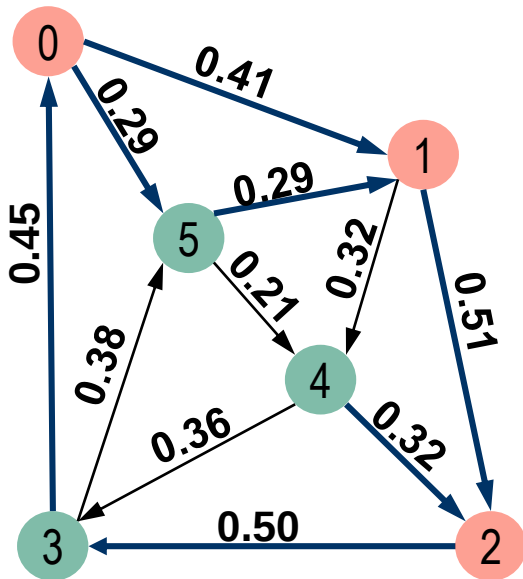
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	∞	0.73	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	∞	0.21	0



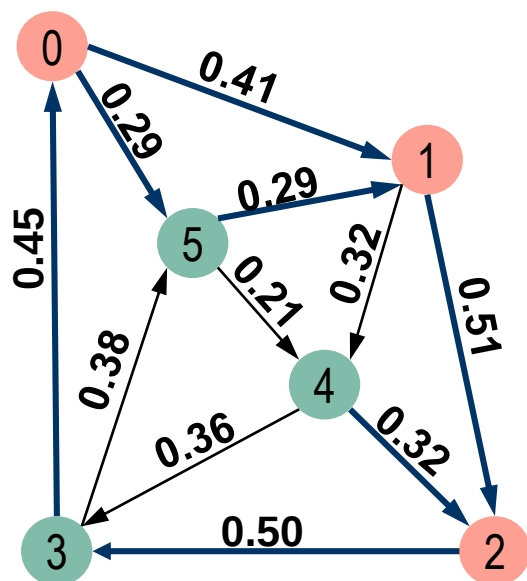
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	∞	0.73	0.29
1	∞	0	0.51	∞	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	∞	0.21	0



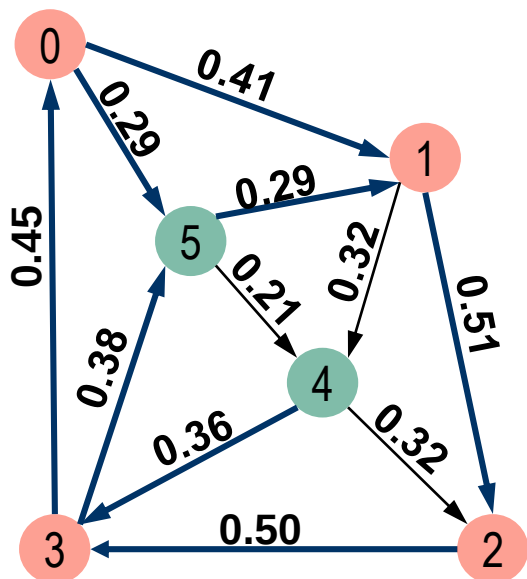
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	∞	0	0.51	1.01	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	1.30	0.21	0



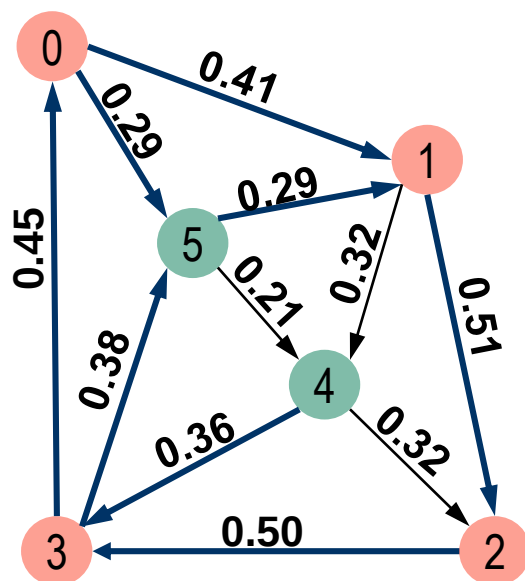
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	∞	0	0.51	1.01	0.32	∞
2	∞	∞	0	0.50	∞	∞
3	0.45	0.86	1.37	0	1.18	0.38
4	∞	∞	0.32	0.36	0	∞
5	∞	0.29	0.80	1.30	0.21	0



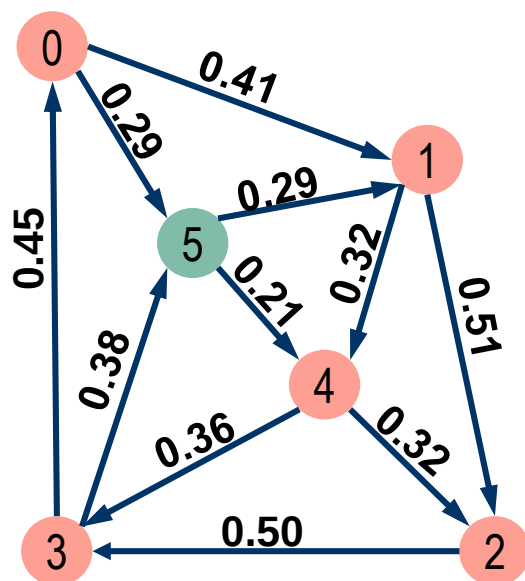
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	1.46	0	0.51	1.01	0.32	1.39
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.75	0.29	0.80	1.30	0.21	0



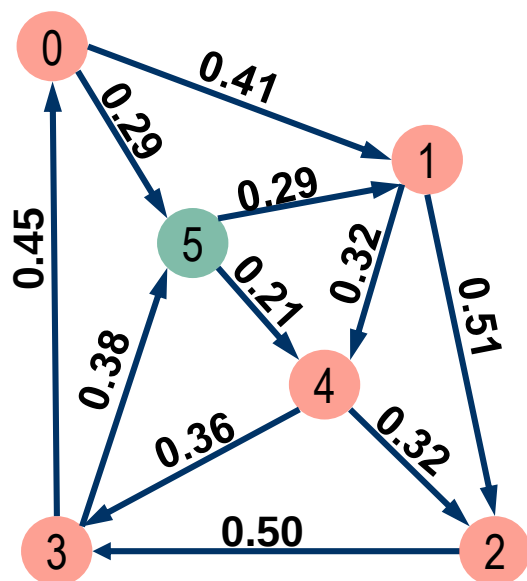
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.42	0.73	0.29
1	1.46	0	0.51	1.01	0.32	1.39
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.75	0.29	0.80	1.30	0.21	0



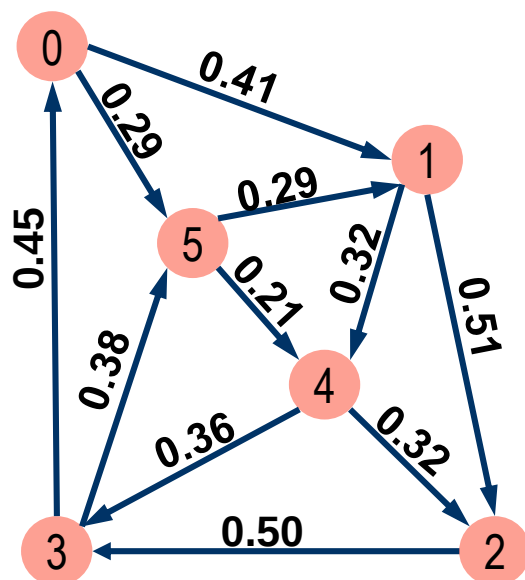
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.09	0.73	0.29
1	1.13	0	0.51	0.68	0.32	1.39
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0



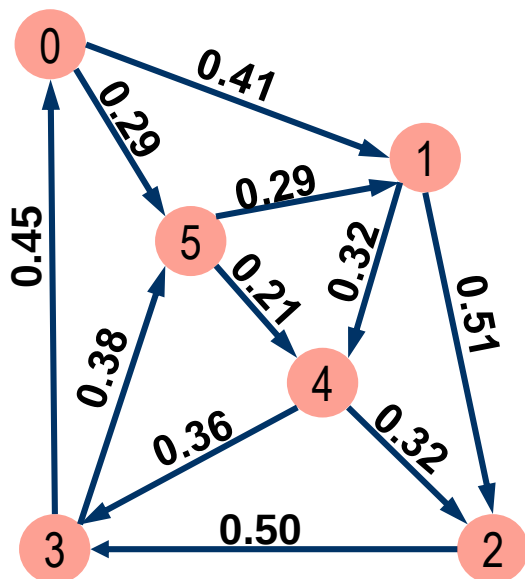
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.92	1.09	0.73	0.29
1	1.13	0	0.51	0.68	0.32	1.39
2	0.95	1.36	0	0.50	1.68	0.88
3	0.45	0.86	1.37	0	1.18	0.38
4	0.81	1.22	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0



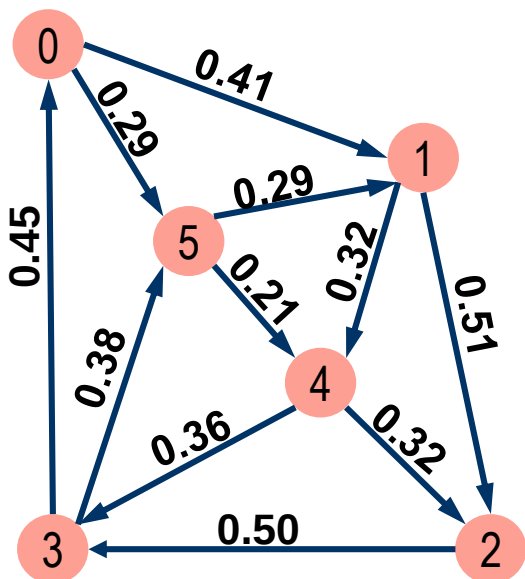
全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	0.41	0.82	0.86	0.50	0.29
1	1.13	0	0.51	0.68	0.32	1.39
2	0.95	1.17	0	0.50	1.09	0.88
3	0.45	0.67	0.91	0	0.69	0.38
4	0.81	1.03	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0



全源最短路径 - Floyd算法

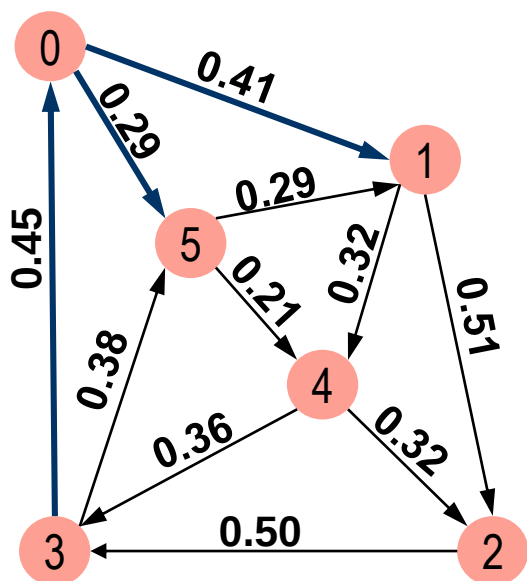


	0	1	2	3	4	5
0	0	0.41	0.82	0.86	0.50	0.29
1	1.13	0	0.51	0.68	0.32	1.39
2	0.95	1.17	0	0.50	1.09	0.88
3	0.45	0.67	0.91	0	0.69	0.38
4	0.81	1.03	0.32	0.36	0	0.74
5	1.02	0.29	0.53	0.57	0.21	0

边值矩阵



全源最短路径 - Floyd算法



	0	1	2	3	4	5
0	0	01	0542	0543	054	05
1	1430	0	12	143	14	1435
2	230	2351	0	23	2354	235
3	30	351	3542	0	354	35
4	430	4351	42	43	0	435
5	5430	51	542	543	54	0

路径矩阵



小结与扩展

➤ 蛮力法 ✓

➤ 分治法

➤ 减治法

➤ 变治法

➤ 贪心法 ✓

➤ 动态规划

信息论创始人香农1952年在贝尔实验室的内部演讲《创造性思维》(Creative Thinking) 中谈到了优秀科研工作者、工程师必须具备的三个特质，以及有助于创造性解决问题的6个思维技巧：**简化，寻找类似问题，以不同形式复述问题，泛化，对问题进行结构化分析，反转问题**

<https://zhuanlan.zhihu.com/p/626036897>

加油同学们!

